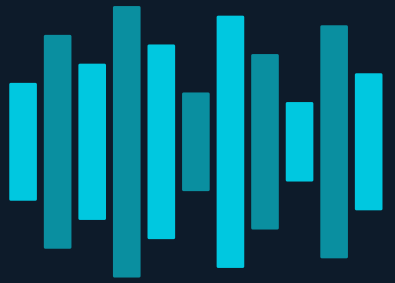
HearEdit

Speaker Diarization System

Milestone 3 – Final Presentation



Agenda

Milestone 3

01 | Project Overview & Recap

02 | Model Pipeline (Vertex AI)

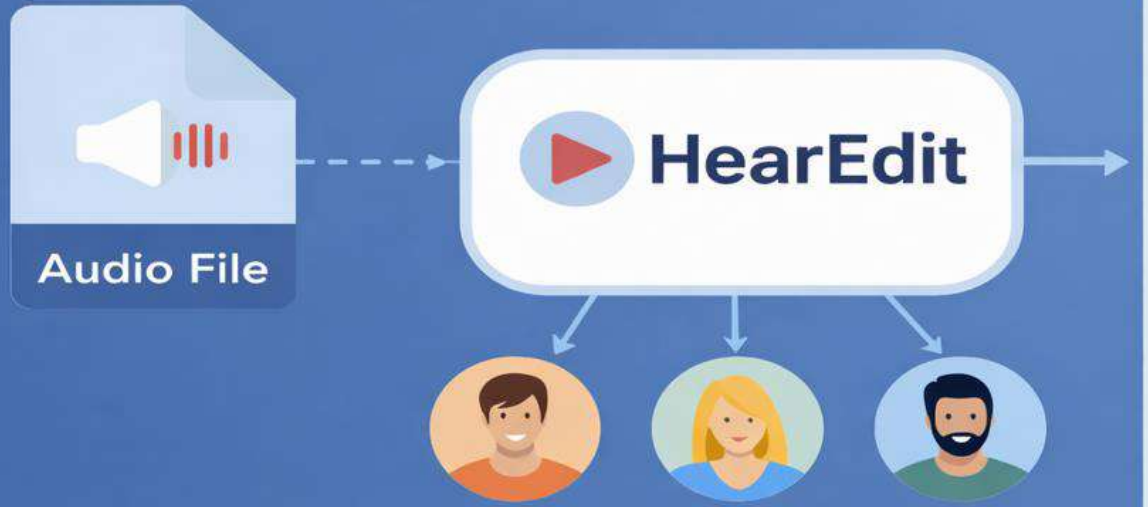
03 | API Design

04 | Dashboard & Demo

05 | CICD & Cloud Deployment

06 | Conclusion & Lessons Learned

1. Project Overview



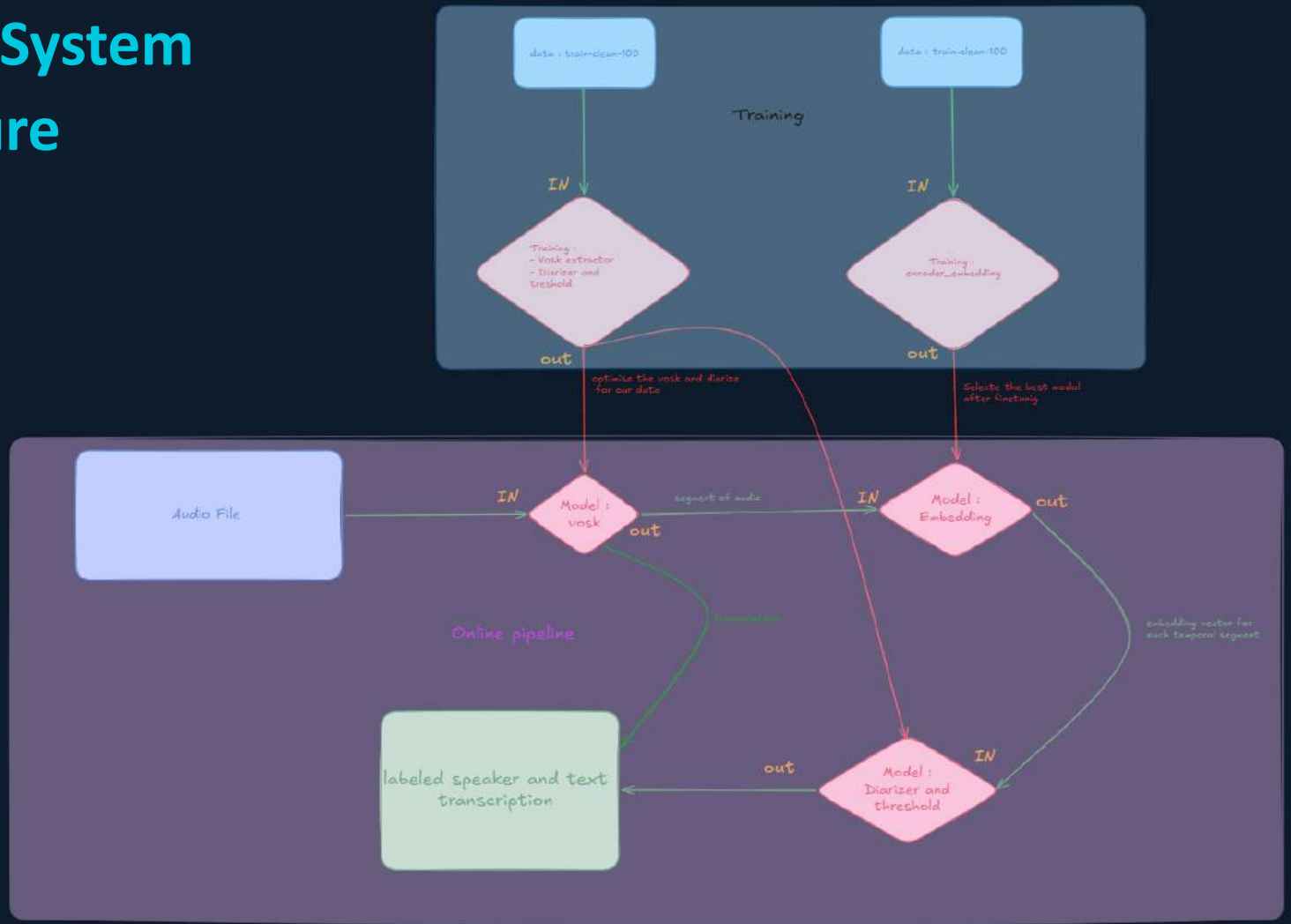
Transcription

Speaker 1
Hey, are we still on for our meeting at 10 AM?

Speaker 2
Yes, I'll be there. Looking forward to it!

Speaker 3
Great! I'll bring the presentation on our latest results.

2. Overall System Architecture



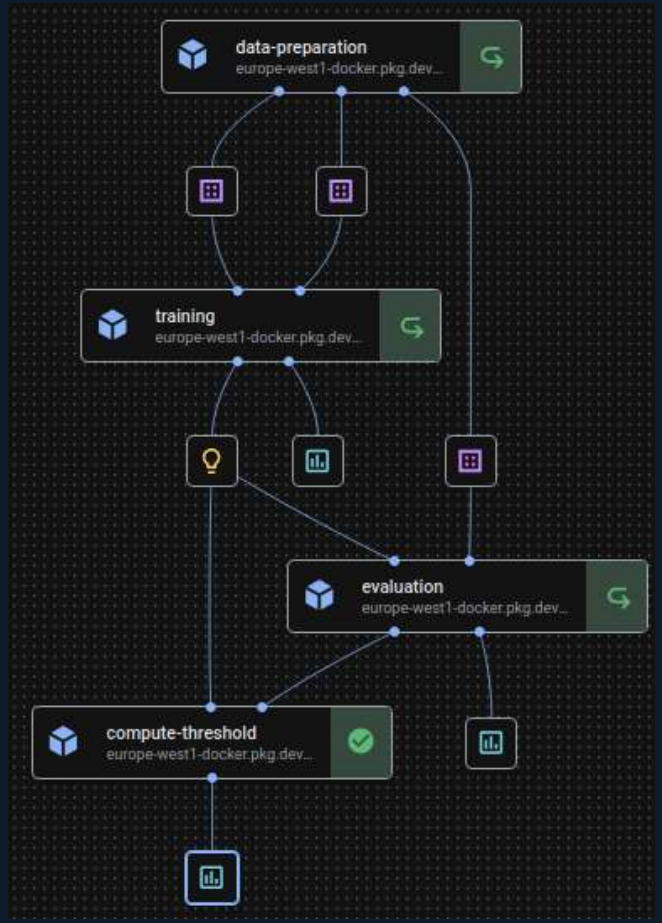
3. Model Pipeline

1. Data Preparation (CPU) Download LibriSpeech from GCS, discover .flac files, split 80/10/10 by speaker → 3 Dataset artifacts (train, val, test)

2. Training (GPU T4, 15 GB RAM) Fine-tune an embedding model (pretrained on VoxCeleb) with ArcFace loss. Saves best model checkpoint based on validation accuracy.

3. Evaluation (CPU) Compute EER and AUC on test split using cosine similarity of embeddings (intra-speaker vs inter-speaker pairs).

4. Compute Threshold (CPU, 16 GB RAM) Recompute EER on a subsample of 3000 files, determine optimal verification threshold, export as JSON to GCS.



4. API Design

ENDPOINTS

POST **/transcribe**
Audio → JSON (segments, timestamps, speakers)

GET **/health**
Cloud Run readiness probe

GET **/past_transcriptions**
In-memory session history

GET **/past_transcriptions/transcription_id**
In-memory session history

BUCKETS

Hearedit-models
- Artifacts
- models
- Pipeline_root

Dataset-bucket
- Dev-clean
- Train-clean-100
- Transcriptions

5. Dashboard

Streamlit Dashboard Features

1. **Upload**
   Drag-and-drop audio file upload (WAV, MP3, M4A)
2. **Live Results**
   Real-time transcription table with speaker labels & timestamps
3. **History**
   Session-based past transcriptions via GET /past_transcriptions
4. **Deployed**
   Publicly accessible on Google Cloud Run (auto-scales)

Serving Mode

Online (Real-time) Serving

- The dashboard sends each audio file directly to the deployed Cloud Run API endpoint (/transcribe).
- Results are returned in real-time and rendered immediately in the Streamlit UI.
- No pre-computed batch results — every request is processed on demand.

Public URL

https://hearedit-api-726024632692.europe-west1.run.app/

6. CI/CD & Cloud Deployment

GitHub Actions Pipeline

Pre-commit hooks

Format checks on every commit
(ruff / black)

Code formatting

Automated lint check in CI
— fails fast on style issues

pytest placeholder

Test stage ready for unit tests
— passes in current state

Auto-deploy (opt.)

CD step pushes Docker image
to Cloud Run on push to main

Gitflow & Repository

feature/* branches

Individual features developed
off develop branch

develop branch

Integration branch, continuous
merge from features

Pull Request

PR from develop → main before
each milestone (reviewers added)

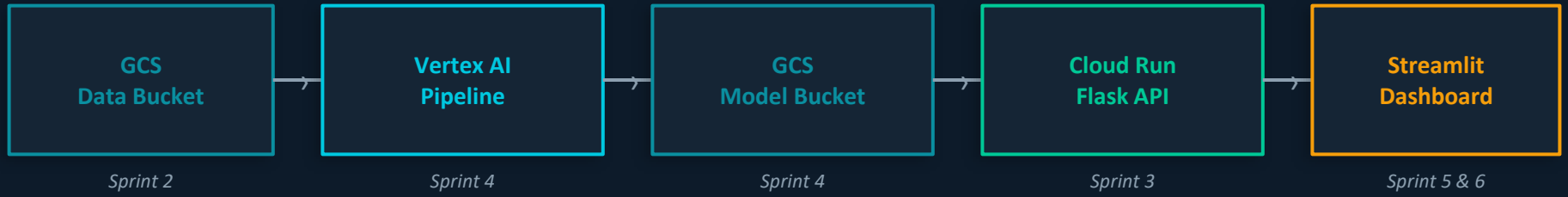
Docs folder

docs/EXPERIMENTATION.md
docs/DEPLOYMENT.md + README

Reviewers

ThomasVrancken + Mapirlet
added as PR reviewers

7. Project view



GitHub Actions CI/CD

pre-commit • lint • pytest • auto-deploy

User uploads audio → Dashboard → API → [Vosk ASR → ECAPA Embed → Diarizer] → JSON transcript → Dashboard renders results

Demonstration

Live demo of HearEdit

1

Upload an audio file to the Streamlit dashboard

2

Watch real-time diarization: speakers identified & labelled

3

Download structured transcript (.txt)

Possible Improvements

1. User uploads an audio file
2. Server sends the first extract of the transcription
3. User corrects the speaker's name, may split the extract, correct the text, etc.
4. User asks for the next extract which is sent
5. repeat step 3-4 until the complete transcription is done
6. The corrections are saved with the audio file
7. On the next training the corrected data are used

Thank You

Questions?